

# Week 2 Instructor Notes — Build and Verify Your Local AI Workbench

Computing Commons

## 1. Build and Verify Your Local AI Workbench

**Purpose:** Set the finish line: evidence, not installation theater.

**Say:** Today we trace a local request, make one bounded change, and prove a result independently. Every check has a limit.

**Ask:** What would count as evidence that the workflow works?

**Do:** Keep a small receipt.

**Watch for:** A green status or generated answer becoming a conclusion.

**Move on when:** Students can name observation, claim, and limit.

## 2. Today's finish line

**Purpose:** Preview trace, narrow change, and independent proof.

**Say:** We practice a repeatable loop rather than mastering every tool.

**Ask:** Which step is easiest to skip?

**Do:** Predict evidence for each outcome.

**Watch for:** Aider action being treated as correctness.

**Move on when:** Students can repeat the evidence ladder.

### 3. Why these tools? The workbench is infrastructure

**Purpose:** Frame tools as shared infrastructure and judgment as human work.

**Say:** Machines provide motion; people provide meaning, scope, and acceptance.

**Ask:** Where does responsibility remain?

**Do:** Connect the rule to debugging or documentation.

**Watch for:** “The tool said so.”

**Move on when:** One human judgment is named.

## 4. The stack: layers, not magic

**Purpose:** Establish request, client, service, model, and machine.

**Say:** A working layer does not prove the next layer. Check the chain in order.

**Ask:** What remains unproven?

**Do:** Name each arrow's observable.

**Watch for:** Aider being called the model.

**Move on when:** Roles are distinct.

## 5. Evidence before model action

**Purpose:** Separate foundation/service observations from inference.

**Say:** python --version proves a runtime answered; ollama list proves availability is reported; loopback proves reachability.

**Ask:** What remains unproven?

**Do:** Write one bounded claim per observation.

**Watch for:** “Installed” becoming “works.”

**Move on when:** All three limits are stated.

## 6. Evidence after model action

**Purpose:** Separate inference and diff evidence.

**Say:** A request proves completion of one request; a diff proves text changed. Neither proves behavior.

**Ask:** What independent check supports behavior?

**Do:** Save tag, response, status, and diff.

**Watch for:** Trusting narration.

**Move on when:** Generation, change, and behavior are distinct.

## 7. A chain of bounded evidence

**Purpose:** Consolidate observation, claim, check, and limit.

**Say:** We build a chain, not reassuring green lights.

**Ask:** What does this prove and not prove?

**Do:** Have one student supply a limit.

**Watch for:** Overclaiming from several observations.

**Move on when:** The chain can be narrated.



## 8. First: open PowerShell

**Purpose:** Make the Windows action recognizable.

**Say:** Find PowerShell 7 and the prompt before typing.

**Ask:** How can you verify the shell?

**Do:** Point to Start, PowerShell 7, and prompt.

**Watch for:** Wrong shell or search box.

**Move on when:** Everyone identifies the prompt.

## 9. Verify PowerShell itself

**Purpose:** Practice command, source, observation, and receipt.

**Say:** The version command answers one narrow question.

**Ask:** What official documentation supports it?

**Do:** Run the command and save smallest output.

**Watch for:** Screenshot substitution.

**Move on when:** Own output and limit are recorded.

## 10. Python: the runtime for our program

**Purpose:** Make the runtime boundary explicit.

**Say:** Version output does not prove packages, program behavior, or tests.

**Ask:** What separate check tests behavior?

**Do:** Run the supplied foundation check.

**Watch for:** Version as validation.

**Move on when:** The untested claim is written.

## 11. Git: the before-and-after witness

**Purpose:** Establish baseline and diff.

**Say:** Git is a witness, not a behavior judge.

**Ask:** Which file and lines changed?

**Do:** Run status and diff.

**Watch for:** Unrelated files or secrets.

**Move on when:** Baseline and intended file are recorded.

## 12. Ollama: a local service on loopback

**Purpose:** Explain locality without overclaiming security.

**Say:** 127.0.0.1 and an answer establish reachability, not inference, privacy, or integration.

**Ask:** What endpoint and response were observed?

**Do:** Run only the approved check.

**Watch for:** Port scans or cloud fallback.

**Move on when:** Reachability and limit are stated.

## 13. Security is a judgment question

**Purpose:** Make safety and security an evidence conversation.

**Say:** Local is a routing fact; security includes configuration, authentication, data handling, and correctness.

**Ask:** Is this safe? Is this secure? How do I know? Where can I verify that?

**Do:** Compare official documentation with local observation.

**Watch for:** Loopback as a blanket privacy guarantee.

**Move on when:** Locality, configuration, and data handling are distinguished.

## 14. The exact approved model: qwen3:8b

**Purpose:** Preserve the exact reviewed boundary.

**Say:** Listed means reported available; inference means one request answered.

**Ask:** What did inference add?

**Do:** Check exact tag and run approved request.

**Watch for:** Substitution or download.

**Move on when:** Tag, command, response, and status are saved.

## 15. Aider is the coding client, not the model

**Purpose:** Clarify client/service/model roles.

**Say:** The approved path is `ollama_chat/qwen3:8b` at `http://127.0.0.1:11434` with `num_ctx: 8192`.

**Ask:** What does `aider -version` not prove?

**Do:** Confirm the local profile only.

**Watch for:** Cloud keys or provider prompts.

**Move on when:** Roles and boundary are named.



## 16. The first Aider win: one bounded change

**Purpose:** Show scope as safety.

**Say:** A good request names one exercise, file, and behavior.

**Ask:** What is the smallest useful request?

**Do:** Record baseline and forbid test edits.

**Watch for:** Scope creep.

**Move on when:** File and behavior are clear.

## 17. Inspect the diff before you trust it

**Purpose:** Place review between generation and acceptance.

**Say:** Aider changed it is not the same as correct.

**Ask:** Any extra file, secret, or cleanup?

**Do:** Exit and inspect the diff.

**Watch for:** Large unreviewed changes.

**Move on when:** Diff is bounded and explainable.

## 18. Independent test: a different claim

**Purpose:** Separate behavior evidence from narration.

**Say:** A passing test supports only its defined behavior.

**Ask:** What does this test exercise?

**Do:** Run it and preserve status/output.

**Watch for:** Partial test as universal proof.

**Move on when:** Coverage boundary is stated.

## 19. Three tiny wins: repeat the loop

**Purpose:** Build confidence through bounded cycles.

**Say:** Independence grows one baseline, ask, inspect, test, and recovery loop at a time.

**Ask:** Which evidence carries forward?

**Do:** Repeat the approved route.

**Watch for:** Increasing scope early.

**Move on when:** All five stages are narrated.

## 20. Recovery is part of the workflow

**Purpose:** Normalize bounded stop and escalation.

**Say:** NOT READY is useful evidence. Stop, preserve, and retry once.

**Ask:** Which failures belong with staff?

**Do:** Practice a failure receipt.

**Watch for:** Switching models or repeated repair.

**Move on when:** A failure receipt exists.

## 21. Prompt structure: from answer to evidence

**Purpose:** Turn vague questions into verifiable requests.

**Say:** Include platform, version, action, observation, source, and limits.

**Ask:** How would you request an independent check?

**Do:** Upgrade one weak question.

**Watch for:** Citation without local verification.

**Move on when:** The revised question has check and limit.

## 22. Learning to fish: the scaffold fades

**Purpose:** Show increasing independence.

**Say:** The scaffold fades; claim, observation, limit, conclusion remain.

**Ask:** What responsibility increases?

**Do:** Place the task on the line.

**Watch for:** Less scaffold becoming less verification.

**Move on when:** One check is defended.

## 23. Work First: the bridge

**Purpose:** Connect the exercise to professional practice.

**Say:** Work First preserves starting state, bounded work, evidence, tests, and durable truth.

**Ask:** Which step protects the next handoff?

**Do:** Map a receipt onto the pipeline.

**Watch for:** Documentation as optional.

**Move on when:** The loop connects to a handoff.



## 24. Show that it works: the Week 2 receipt

**Purpose:** Define observable completion.

**Say:** A small receipt beats a confident story.

**Ask:** What is missing?

**Do:** Assemble and read receipts.

**Watch for:** Missing failure evidence or exact model.

**Move on when:** Change and test are demonstrated.

## 25. Your turn

**Purpose:** Hand off to student work.

**Say:** Ask narrowly, inspect honestly, test independently, and keep the receipt.

**Ask:** What does this prove?

**Do:** Release the supplied exercise.

**Watch for:** Wrong folder or boundary changes.

**Move on when:** Baseline receipt has begun.